

## ***Bases de Dados – Prof. Eduardo Corrêa***

### ***Aula 17 – SQL: Recursos Úteis para Exploração e Preparação de Dados***

Esta aula cobre funcionalidades da Linguagem SQL muito úteis para os cientistas de dados executarem tarefas de exploração e preparação de dados:

- **Single-Row Functions:** também conhecidas como **funções escalares**, estas funções retornam **um único resultado para cada linha** de uma consulta.
- **LIMIT:** cláusula utilizada para **limitar os resultados** (quantidade de linhas) retornados por um SELECT.
- **Estrutura CASE / WHEN:** estrutura que funciona de forma **equivalente ao CASE das linguagens de programação**.
- **LIKE:** operador utilizado na cláusula WHERE para permitir a **busca por um padrão específico** em uma coluna. Apenas **padrões simples** podem ser especificados.

As seções a seguir apresentam exemplos práticos de utilização destas funcionalidades com a utilização da base de dados da “TopicosAdicionais.db”. Esta base de dados possui apenas uma tabela, que armazena informações sobre 100 filmes produzidos em diferentes épocas e países:

```
t_filme(  
  id: INT,                --chave primária (surrogate key)  
  titulo: VARCHAR(100),   --título do filme  
  ano: INT,               --ano de produção  
  resumo: VARCHAR(4000),  --resumo do filme em Português  
  pais: CHAR(2)           --sigla do país com 2 letras  
  duracao: INT,          --duração em minutos  
  avaliacao: NUM         --avaliação (mínimo 0.0, máximo 5.0).  
)
```

#### **1.1. Funções “Single-Row”**

Os principais SGBDs comerciais e livres costumam oferecer uma rica biblioteca de funções *single-row* para seus usuários. O nome se deve ao fato deste tipo de função atuar sobre uma única linha por vez e retornar um resultado por linha (ao contrário do que ocorre com as funções de grupo - como MAX, MIN, AVG e COUNT - que atuam sobre grupos de linha e retornam um resultado por grupo).

Uma função *single-row* trabalha da seguinte forma: ela recebe um ou mais parâmetros como entrada, realiza algum processamento interno sobre esses parâmetros e retorna um valor único como saída. Os exemplos a seguir introduzem algumas das

principais funções *single-row* oferecidas pelo SQLite. Nesta aula, cobriremos apenas as funções de caractere (utilizadas para manipular texto), as funções numéricas (utilizadas para manipular números inteiros ou reais) e a função CAST para conversão de tipos. As funções para tratamento de colunas do tipo DATA/HORA serão cobertas em uma aula futura, onde abordaremos os tipos DATE e TIMESTAMP.

## 2.1 Funções de Caractere

Também chamadas de funções de *string*, são utilizadas para manipular colunas alfanuméricas, como CHAR, VARCHAR, TEXT, etc.

### LENGTH

**Sintaxe:** LENGTH(s)

**Descrição:** recebe como parâmetro uma string *s* e retorna o seu comprimento (medido em número de caracteres, incluindo os espaços em branco).

**Exemplo:** retorna o título do filme, ano, resumo e comprimento do resumo, ordenando do maior comprimento para o menor:

```
SELECT titulo, ano, resumo, LENGTH(resumo)
FROM t_filme
ORDER BY LENGTH(resumo) DESC
```

| titulo                       | ano  | resumo                      | LENGTH(resumo) |
|------------------------------|------|-----------------------------|----------------|
| ... E o Vento Levou          | 1939 | Uma reunião social acon...  | 3317           |
| Coisas Belas e Sujas         | 2002 | Em Londres um nigeriano...  | 1200           |
| Para o Resto de Nossas Vidas | 1992 | Há 10 anos atrás eles f...  | 1184           |
| Os Incompreendidos           | 1959 | Antoine Doinel (Jean-Pi...  | 1182           |
| Dança Comigo?                | 1996 | Shohei Sugiyama (Kôji Y...  | 977            |
| O Filho da Noiva             | 2001 | Aos 42 anos Rafael Belv...  | 867            |
| Fanny e Alexander            | 1982 | No início do século XX, ... | 827            |
| Xingu                        | 2011 | Três irmãos decidem vive... | 826            |
| ...                          |      |                             |                |

### LOWER e UPPER

**Sintaxe:** LOWER(s) e UPPER(s)

**Descrição:** as funções LOWER e UPPER recebem como parâmetro uma string *s* e retornam os seus caracteres ASCII convertidos para minúsculo e maiúsculo, respectivamente.

**Exemplo:** retorna o título, título em minúsculo e o título em maiúsculo de todos os filmes.

```
SELECT titulo, LOWER(titulo), UPPER(titulo)
FROM t_filme WHERE ano=2016
```

| titulo                        | LOWER(titulo)                 | UPPER(titulo)                 |
|-------------------------------|-------------------------------|-------------------------------|
| La La Land: Cantando Estações | la la land: cantando estações | LA LA LAND: CANTANDO ESTAÇÕES |

Observe que os caracteres acentuados não são convertidos (pois eles não fazem parte da tabela ASCII<sup>1</sup>). Felizmente, os SGBDs mais sofisticados não costumam ter esse tipo de limitação.

## INSTR

Sintaxe: INSTR(*s*, *sub*)

Descrição: fornece a localização da substring especificada no parâmetro *sub* dentro da string *s*. Retorna 0 caso *sub* não faça parte de *s*.

Exemplo: retorna a posição da palavra “Bennet” nos resumos dos filmes de 2005.

```
SELECT titulo, resumo, INSTR(resumo, 'Bennet')
FROM t_filme WHERE ano=2005
```

| titulo                | resumo(titulo)                              | SUBSTR(resumo, 'Bennet') |
|-----------------------|---------------------------------------------|--------------------------|
| Orgulho e Preconceito | Inglaterra, 1797. As cinco irmãs Bennet ... | 34                       |
| O que Você Faria?     | Sete executivos disputam uma única vaga ... | 0                        |
| Meu Melhor Amigo      | François Coste (Daniel Auteuil) é um com... | 0                        |
| ...                   |                                             |                          |

## REPLACE

Sintaxe: REPLACE(*s*, *from*, *to*)

Descrição: modifica todas as ocorrências da substring *from* pela substring *to* na string *s*.

Exemplo: troca a string ‘Orgulho’ por ‘Vaidade’ em todos os filmes que possuem ‘Orgulho’ no título.

```
SELECT titulo, REPLACE(titulo, 'Orgulho', 'Vaidade')
FROM t_filme WHERE INSTR(titulo, 'Orgulho') > 0
```

| titulo                | REPLACE(titulo, 'Orgulho', 'Vaidade') |
|-----------------------|---------------------------------------|
| Orgulho e Preconceito | Vaidade e Preconceito                 |

O exemplo anterior mostra que também podemos utilizar qualquer função na cláusula WHERE, como foi feito com a função INSTR.

---

<sup>1</sup>O manual do SQLite explica o motivo da adoção desta abordagem: <http://www.sqlite.org/faq.html#q18>. De qualquer forma, é possível contornar essa característica do SQLite com o uso de configurações customizadas ou através da criação da sua própria função de conversão.

## TRIM, LTRIM, RTRIM

**Sintaxe:** TRIM(*s*, *c*), LTRIM(*s*, *c*), RTRIM(*s*, *c*)

**Descrição:** A função TRIM remove todas as ocorrências do caractere *c* de ambos os lados da string *s* (caso *c* seja omitido, remove os espaços em branco). LTRIM faz a remoção apenas no lado esquerdo e RTRIM no direito.

**Exemplo:** remove o caractere 'A' do lado esquerdo de todos os filmes que começam com 'A ' (letra 'A' seguida de espaço em branco).

```
SELECT titulo, LTRIM(titulo, 'A') FROM
t_filme WHERE titulo LIKE 'A %'
```

| titulo                        | LTRIM(titulo, 'A')          |
|-------------------------------|-----------------------------|
| A Copa                        | Copa                        |
| A Família Bélier              | Família Bélier              |
| A Hora da Estrela             | Hora da Estrela             |
| A Insustentável Leveza do Ser | Insustentável Leveza do Ser |
| A Onda                        | Onda                        |
| A Rosa Púrpura do Cairo       | Rosa Púrpura do Cairo       |
| A Vida é Bela                 | Vida é Bela                 |

No exemplo acima, a simples remoção de 'A' faz com que os títulos passem a ser iniciados por espaço. Você pode combinar TRIM e LTRIM para remover esse espaço extra.

```
SELECT titulo, TRIM(LTRIM(titulo, 'A')) FROM
t_filme WHERE titulo LIKE 'A %'
```

| titulo                        | LTRIM(titulo, 'A')          |
|-------------------------------|-----------------------------|
| A Copa                        | Copa                        |
| A Família Bélier              | Família Bélier              |
| A Hora da Estrela             | Hora da Estrela             |
| A Insustentável Leveza do Ser | Insustentável Leveza do Ser |
| A Onda                        | Onda                        |
| A Rosa Púrpura do Cairo       | Rosa Púrpura do Cairo       |
| A Vida é Bela                 | Vida é Bela                 |

## SUBSTR

**Sintaxe:** SUBSTR(*s*, *pos*, *tam*)

**Descrição:** Retorna a substring de *s* que inicia na posição *pos* e tem comprimento *tam*.

**Exemplo:** Exibir os 30 primeiros caracteres dos resumos.

```
SELECT titulo, SUBSTR(resumo, 1, 30)
FROM t_filme WHERE titulo LIKE 'A %'
```

| titulo                        | SUBSTR(resumo, 1, 30)          |
|-------------------------------|--------------------------------|
| A Insustentável Leveza do Ser | Nos anos 60 em Praga, Tchecosl |

|                         |                                |
|-------------------------|--------------------------------|
| A Copa                  | Orgyen é um jovem monge que te |
| A Família Bélier        | Paula é uma adolescente france |
| A Vida é Bela           | Durante a Segunda Guerra Mundi |
| A Onda                  | Em uma escola da Alemanha, alu |
| A Hora da Estrela       | Macabéa (Marcélia Cartaxo) é u |
| A Rosa Púrpura do Cairo | Em área pobre de Nova Jersey,  |

## CONCATENAÇÃO

**Sintaxe:** ||

**Descrição:** Na verdade a concatenação não é uma função, mas um operador que pode ser aplicado a valores alfanuméricos, servindo para combinar duas ou mais strings em uma só.

**Exemplo:** Exibir os 30 primeiros caracteres de cada resumo concatenado com '...':

```
SELECT titulo, SUBSTR(resumo, 1, 30) || '...'
FROM t_filme WHERE titulo LIKE 'A %'
```

| titulo                        | SUBSTR(resumo,1,30)    '...'      |
|-------------------------------|-----------------------------------|
| A Insustentável Leveza do Ser | Nos anos 60 em Praga, Tchecosl... |
| A Copa                        | Orgyen é um jovem monge que te... |
| A Família Bélier              | Paula é uma adolescente france... |
| A Vida é Bela                 | Durante a Segunda Guerra Mundi... |
| A Onda                        | Em uma escola da Alemanha, alu... |
| A Hora da Estrela             | Macabéa (Marcélia Cartaxo) é u... |
| A Rosa Púrpura do Cairo       | Em área pobre de Nova Jersey, ... |

## 1.2 Funções de Conversão de Tipo

### CAST

**Sintaxe:** CAST(*t1* AS *t2*)

**Descrição:** Converte um valor do tipo *t1* para o tipo *t2* (ex.: de INT para FLOAT, de VARCHAR para INT, etc.).

**Exemplo:** Exibir a duração de um filme em horas (está originalmente armazenada em minutos).

```
SELECT titulo, ano, duracao AS dur_min, CAST(duracao AS FLOAT) / 60 AS dur_hor
FROM t_filme ORDER BY duracao DESC
```

| titulo                        | ano  | dur_min | dur_hor           |
|-------------------------------|------|---------|-------------------|
| ... E o Vento Levou           | 1939 | 238     | 3.966666666666667 |
| Gandhi                        | 1982 | 190     | 3.166666666666667 |
| Fanny e Alexander             | 1982 | 188     | 3.133333333333333 |
| Amadeus                       | 1984 | 180     | 3                 |
| A Insustentável Leveza do Ser | 1988 | 171     | 2.85              |
| ...                           |      |         |                   |

O CAST é necessário nesse exemplo porque “duracao” é um campo INT. Por default, o SQLite **retorna apenas a parte inteira da divisão** sempre que o operador “/” for utilizado sobre um campo do **tipo inteiro**. Esse comportamento não é comum nos outros SGBDs, mas sabemos que o SQLite possui algumas peculiaridades... Veja o que acontece quando o CAST não é usado:

```
SELECT titulo, ano, duracao AS dur_min, duracao / 60 AS dur_hor
FROM t_filme ORDER BY duracao DESC
```

| titulo                        | ano  | dur_min | dur_hor |
|-------------------------------|------|---------|---------|
| ... E o Vento Levou           | 1939 | 238     | 3       |
| Gandhi                        | 1982 | 190     | 3       |
| Fanny e Alexander             | 1982 | 188     | 3       |
| Amadeus                       | 1984 | 180     | 3       |
| A Insustentável Leveza do Ser | 1988 | 171     | 2       |
| ...                           |      |         |         |

## COALESCE

Sintaxe: COALESCE(*x*, *valor*)

Descrição: caso o valor de *x* for NULL, será trocado por *valor*<sup>2</sup>.

**Exemplo**: o exemplo abaixo trocará NULL por 0, sempre que o valor de número for NULL.

```
SELECT COALESCE(numero, 0) FROM
(SELECT 1000 AS numero
 UNION
 SELECT NULL AS numero
 UNION
 SELECT 2000 AS numero
 ) t
ORDER BY numero
```

```
COALESCE(numero, 0)
0
1000
2000
```

## 1.3 Funções Numéricas

### ROUND

Sintaxe: ROUND(*n*, *d*)

---

<sup>2</sup> O exemplo apresenta apenas a sintaxe básica do COALESCE. Consulte o manual do SQLite para outras formas de utilização.

Descrição: Arredonda o número  $n$  para um certo número de casas decimais, especificado no parâmetro  $d$ . Caso  $d$  seja omitido, arredonda para o inteiro mais próximo (se o valor da parte decimal é maior ou igual a .5, arredonda pra cima; senão, arredonda para baixo).

Exemplo: Exibir a duração de um filme em horas, com duas casas decimais.

```
SELECT titulo, ano, duracao AS dur_min,
ROUND(CAST(duracao AS FLOAT) / 60,2) AS dur_hor
FROM t_filme ORDER BY duracao DESC
```

| titulo                        | ano  | dur_min | dur_hor |
|-------------------------------|------|---------|---------|
| ... E o Vento Levou           | 1939 | 238     | 3.97    |
| Gandhi                        | 1982 | 190     | 3.17    |
| Fanny e Alexander             | 1982 | 188     | 3.13    |
| Amadeus                       | 1984 | 180     | 3       |
| A Insustentável Leveza do Ser | 1988 | 171     | 2.85    |
| O Tambor                      | 1979 | 162     | 2.7     |
| Carandiru                     | 2003 | 146     | 2.43    |
| Julietta dos Espíritos        | 1965 | 137     | 2.28    |
| ...                           |      |         |         |

## RANDOM

Sintaxe: RANDOM()

Descrição: Retorna um valor aleatório entre -9223372036854775808 e +9223372036854775807

Exemplo:

```
SELECT RANDOM() --o seu resultado será diferente...
-6785023889944559713
```

Veja que tanto neste exemplo, como na subconsulta do exemplo envolvendo a função COALESCE, fizemos uso do SELECT sem FROM! Isso é permitido no SQLite e em alguns outros SGBDs, em situações onde deseja-se obter um resultado que não vem de nenhuma tabela.

## ABS

Sintaxe: ABS()

Descrição: Retorna o valor absoluto (módulo) de um número

Exemplo:

```
SELECT ABS(1000), ABS(-500), ABS(2 * -2.5), ABS(0)
ABS(1000)    ABS(-500)    ABS(2 * -2.5)    ABS(0)
1000         500         5                 0
```

## 1.4 Exemplos Finais

A seguir um exemplo onde diferentes funções são utilizadas nas cláusulas SELECT, FROM e ORDER BY. O exemplo lista o título, ano, resumo e comprimento do resumo de todos os filmes que possuem 'Rio de Janeiro' no resumo (independente de estar escrita em letras maiúsculas ou minúsculas), ordenando os resultados pelo comprimento do resumo.

```
SELECT titulo, ano, resumo, LENGTH(resumo) AS compr_resumo
FROM t_filme
WHERE INSTR(UPPER(resumo), 'RIO DE JANEIRO') > 0
ORDER BY LENGTH(resumo)
```

| titulo                         | ano  | resumo                  | compr_resumo |
|--------------------------------|------|-------------------------|--------------|
| Central do Brasil              | 1998 | Dora (Fernanda Monte... | 407          |
| Onde Está Você, João Gilberto? | 2018 | Inspirado no livro H... | 447          |
| Se Segura, Malandro!           | 1978 | Rio de Janeiro. Paul... | 479          |
| Nise - O Coração da Loucura    | 2015 | Ao voltar a trabalha... | 486          |
| Elis                           | 2015 | Cantora desde a infâ... | 503          |
| Leila Diniz                    | 1987 | Biografia da artista... | 558          |
| Gonzaga: de Pai para Filho     | 2012 | Decidido a mudar seu... | 820          |

O exemplo a seguir combina o uso de uma função de grupo com uma função *single-row*. O exemplo mostra a média das avaliações e o total de filmes agrupado por país, utilizando ROUND na média para arredondar os resultados para uma casa decimal:

```
SELECT pais, ROUND(AVG(avaliacao),1) AS nota_media, COUNT(*) AS tot_filmes
FROM t_filme
GROUP BY pais
ORDER BY 2 DESC, 1
```

| pais | nota_media | tot_filmes |
|------|------------|------------|
| CL   | 4.2        | 1          |
| UK   | 4.1        | 18         |
| AR   | 4          | 5          |
| IT   | 4          | 3          |
| US   | 4          | 17         |
| UY   | 4          | 2          |
| CA   | 3.9        | 2          |
| FR   | 3.9        | 10         |
| BR   | 3.8        | 24         |
| DE   | 3.8        | 5          |
| ES   | 3.8        | 5          |
| IN   | 3.8        | 1          |
| SE   | 3.8        | 2          |
| BE   | 3.7        | 1          |
| JP   | 3.4        | 1          |
| BT   | 3.2        | 1          |
| CU   | 3.2        | 1          |
| KZ   | 3.1        | 1          |

### 1.5 Cláusula LIMIT

Esta cláusula é utilizada para limitar a quantidade de resultados retornados por um comando SELECT. Sua sintaxe básica é apresentada abaixo. O WHERE e ORDER BY são opcionais. Veja que o LIMIT deve ser especificado após o ORDER BY.

#### Sintaxe:

```
SELECT colunas
FROM tabela
[WHERE]
[ORDER BY]
LIMIT número de linhas
```

**Exemplo:** Retornar apenas os 8 primeiros filmes produzidos entre 1970 e 1979.

```
SELECT titulo, ano
FROM t_filme
WHERE ano BETWEEN 1970 AND 1979
ORDER BY ano
LIMIT 8
```

| titulo                                  | ano  |
|-----------------------------------------|------|
| Ensina-me a Viver                       | 1971 |
| O Discreto Charme da Burguesia          | 1972 |
| Monty Python em Busca do Cálice Sagrado | 1975 |
| Se Segura, Malandro!                    | 1978 |
| Bye Bye Brasil                          | 1979 |
| Hair                                    | 1979 |
| Mamãe Faz 100 Anos                      | 1979 |
| O Casamento de Maria Braun              | 1979 |

A cláusula OFFSET pode ser utilizada para determinar qual será o primeiro resultado apresentado por LIMIT:

```
SELECT colunas
FROM tabela
[WHERE]
[ORDER BY]
LIMIT número de linhas OFFSET valor
```

**Exemplo:** Retornar apenas os 8 primeiros filmes produzidos entre 1970 e 1979, iniciando a partir da quarta linha (não mostrará mais “Ensina-me a Viver”, “Estado de Sítio” e “O Discreto Charme da Burguesia” - o uso de OFFSET 3 faz com que 3 linhas sejam puladas).

```
SELECT titulo, ano
FROM t_filme
WHERE ano BETWEEN 1970 AND 1979
ORDER BY ano
LIMIT 8 OFFSET 3
```

| <b>titulo</b>              | <b>ano</b> |
|----------------------------|------------|
| Se Segura, Malandro!       | 1978       |
| Bye Bye Brasil             | 1979       |
| Hair                       | 1979       |
| Mamãe Faz 100 Anos         | 1979       |
| O Casamento de Maria Braun | 1979       |
| O Tambor                   | 1979       |

## 1.6. CASE ... WHEN

A estrutura CASE ... WHEN permite com que uma série de condições seja avaliada, retornando um valor quando a primeira condição for alcançada. É similar ao IF-THEN-ELSE ou o CASE das linguagens de programação.

### Sintaxe:

```
CASE [expressão]
  WHEN condição_1 THEN resultado_1
  WHEN condição_2 THEN resultado_2
  ...
  WHEN condição_n THEN resultado_n
  ELSE resultado
END
```

Esta estrutura é especialmente útil quando desejamos converter valores numéricos em categorias, como mostra o exemplo a seguir.

**Exemplo:** retorna o conceito (A+, A, A- ou B) de um filme, que é determinado em função de sua nota de avaliação.

```
SELECT titulo, ano, avaliacao,
CASE
  WHEN avaliacao BETWEEN 4.4 AND 5.0 THEN 'A+'
  WHEN avaliacao BETWEEN 4.0 AND 4.3 THEN 'A'
  WHEN avaliacao BETWEEN 3.5 AND 3.9 THEN 'A-'
  ELSE 'B'
END AS conceito
FROM t_filme ORDER BY titulo
```

| <b>titulo</b>                 | <b>ano</b> | <b>avaliação</b> | <b>conceito</b> |
|-------------------------------|------------|------------------|-----------------|
| ... E o Vento Levou           | 1939       | 4.5              | A+              |
| A Copa                        | 1999       | 3.2              | B               |
| A Família Bélier              | 2014       | 4.4              | A+              |
| A Hora da Estrela             | 1985       | 3.6              | A-              |
| A Insustentável Leveza do Ser | 1988       | 3.9              | A-              |
| A Onda                        | 2008       | 4.5              | A+              |
| A Rosa Púrpura do Cairo       | 1985       | 4                | A               |
| A Vida é Bela                 | 1997       | 4.7              | A+              |
| ...                           |            |                  |                 |

### 1.7. Operador LIKE

O operador LIKE é utilizado para “casar” valores texto contra um padrão que é determinado através das chamadas *wildcards*. O LIKE é bastante simples de usar, pois existem apenas duas *wildcards*:

- % : o sinal de percentual representa zero, um ou múltiplos caracteres.
- \_ : O *underscore* representa um único caractere.

Apesar de sua simplicidade (ou talvez por causa dela!), o LIKE é um comando muito utilizado e que funciona bem para um grande número de situações práticas.

**Exemplo:** retornar todos os filmes cujo título começa com M:

```
SELECT titulo, ano FROM t_filme WHERE titulo LIKE 'M%'
```

| titulo                                  | ano  |
|-----------------------------------------|------|
| Macunaíma                               | 1969 |
| Mamãe Faz 100 Anos                      | 1979 |
| Meu Melhor Amigo                        | 2005 |
| Monty Python em Busca do Cálice Sagrado | 1975 |
| Morango e Chocolate                     | 1993 |

**Exemplo:** retornar todos os filmes cujo título termina com ‘as’:

```
SELECT titulo, ano FROM t_filme WHERE titulo LIKE '%as'
```

| titulo                                     | ano  |
|--------------------------------------------|------|
| As Invasões Bárbaras                       | 2003 |
| Coisas Belas e Sujas                       | 2002 |
| Nove Rainhas                               | 2000 |
| Para o Resto de Nossas Vidas               | 1992 |
| Peixe Grande e suas Histórias Maravilhosas | 2003 |

**Exemplo:** retornar todos os filmes que possuem ‘RIO DE JANEIRO’ em qualquer posição do resumo:

```
SELECT titulo, ano FROM t_filme  
WHERE UPPER(resumo) LIKE '%RIO DE JANEIRO%'
```

| titulo                         | ano  |
|--------------------------------|------|
| Nise - O Coração da Loucura    | 2015 |
| Se Segura, Malandro!           | 1978 |
| Leila Diniz                    | 1987 |
| Elis                           | 2015 |
| Central do Brasil              | 1998 |
| Gonzaga: de Pai para Filho     | 2012 |
| Onde Está Você, João Gilberto? | 2018 |

**Exemplo:** retornar todos os filmes que possuem a letra “u” na segunda posição do título:

```
SELECT titulo, ano FROM t_filme WHERE titulo LIKE '_u%'
```

| titulo                       | ano  |
|------------------------------|------|
| Eu, Daniel Blake             | 2017 |
| Julietta dos Espíritos       | 1965 |
| Lunchbox                     | 2013 |
| Qual é o Nome do Bebê?       | 2012 |
| Que Horas Ela Volta?         | 2015 |
| Quem Quer Ser um Milionário? | 2008 |
| Tudo Sobre Minha Mãe         | 1999 |
| Tulpan                       | 2010 |

**Observação:** Ao contrário do que ocorre quando usamos as funções de caractere, o SQLite não faz distinção entre caracteres maiúsculos e minúsculos no LIKE (algo diferente do que ocorre na maioria dos outros SGBDs). Isto quer dizer que se você fizer LIKE '%A' ou LIKE '%a' ele achará os mesmos resultados! Para modificar esse comportamento é preciso alterar o valor do parâmetro “case\_sensitive\_like”:

```
PRAGMA case_sensitive_like=ON;  --LIKE passa a distinguir caracteres  
                                --minúsculos e maiúsculos
```

```
PRAGMA case_sensitive_like=OFF; --LIKE passa a não distinguir caracteres  
                                --minúsculos e maiúsculos
```

### EXPRESSÕES REGULARES

De uma maneira simplificada, podemos dizer que as expressões regulares possuem o mesmo propósito do operador LIKE: permitir a especificação de um padrão para ser “casado” com valores de um texto. No entanto, as expressões regulares possibilitam com que padrões muito mais complexos possam ser definidos. Trata-se de uma tecnologia tão popular e importante que está disponível não apenas na SQL, mas praticamente em todas as linguagens de programação, incluindo R e Python.

No campo da Estatística, as expressões regulares são especialmente úteis para aqueles que desejam trabalhar em projetos envolvendo Mineração de Texto e Web Scraping. O SQLite suporta o uso de expressões regulares através do operador REGEXP. Apenas para você ter uma ideia, o SELECT abaixo retorna todos os filmes que possuem algum número aparecendo em seu título (seja qual for o número!):

```
SELECT titulo, ano FROM t_filme WHERE titulo REGEXP '[0-9]'
```

| titulo               | ano  |
|----------------------|------|
| Fahrenheit 451       | 1966 |
| Mamãe Faz 100 Anos   | 1979 |
| Uma Noite de 12 Anos | 2018 |

Para conhecer sobre o assunto, recomendamos o excelente tutorial disponibilizado em: <http://aurelio.net/regex/guia/>.